

Industrial Internship Report

on

Banking Information System

Prepared by

SAI YOGESH

Executive Summary

This report presents the work completed during the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT). The internship focused on applying Core Java programming concepts to a practical problem statement.

The project developed for this report is a console-based **Banking Information System**. It provides basic banking operations such as account creation, login, deposit, withdrawal, money transfer, balance enquiry and transaction history.

The project helped demonstrate object-oriented programming, collections, exception handling, input validation, modular program design and problem-solving. The application is intentionally implemented as a local educational prototype and does not represent a production banking system.

TABLE OF CONTENTS

1	Preface	3
2	Introduction	4
2.1	About UniConverge Technologies Pvt Ltd	4
2.2	About upskill Campus	8
2.3	The IoT Academy	10
2.4	Objectives of this Internship Program	10
2.5	Reference	10
2.6	Glossary	10
3	Problem Statement	11
4	Existing and Proposed Solution	12
4.1	Code Submission (GitHub Link)	12
4.2	Report Submission (GitHub Link)	12
5	Proposed Design / Model	13
5.1	High Level Design	13
5.2	Low Level Design	13
5.3	Interfaces	13
6	Performance Test	14
6.1	Test Plan / Test Cases	14
6.2	Test Procedure	14
6.3	Performance Outcome	14
7	My Learnings	15
8	Future Work Scope	16

1 Preface

The six-week internship provided an opportunity to connect Core Java concepts with a practical application. The work was organized around understanding the problem, planning the solution, implementing the program, validating the implementation and documenting the outcome.

The Banking Information System was selected as the project implementation. The application models common banking operations through a menu-driven console interface. The focus was on creating a clear and understandable Java program rather than claiming production-level banking security.

Program flow: Explore problem statement → Plan solution → Implement modules → Validate operations → Test edge cases → Prepare report and submission.

The internship strengthened my understanding of object-oriented design, Java collections, input validation and structured problem solving. It also highlighted the difference between an academic prototype and an industrial system, especially in areas such as authentication, persistence, security and scalability.

Acknowledgement: I thank the internship organizers, mentors, faculty members and everyone who supported the completion of the project. Their guidance helped me understand how a problem can be converted into a working software solution.

Message to juniors and peers: Treat the project as an opportunity to understand the complete development cycle. Do not focus only on getting the program to run; test invalid inputs, document design decisions and understand the limitations of the solution.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

According to the supplied internship report template, UniConverge Technologies Pvt Ltd (UCT) was established in 2013 and works in the Digital Transformation domain, with focus on industrial solutions, sustainability and RoI. The template identifies technologies used by UCT including Internet of Things (IoT), Cyber Security, Cloud Computing, Machine Learning, communication technologies such as 4G/5G/LoraWAN, Java Full Stack, Python and front-end technologies.

The template also describes UCT Insight as an IoT platform with Java used for the backend and ReactJS for the front end, with support for MySQL and NoSQL databases. It mentions protocols such as MQTT, CoAP, HTTP, Modbus TCP and OPC UA, and both cloud and on-premises deployment.

The same source describes Factory Watch as a smart-factory platform supporting production and asset monitoring, OEE, predictive maintenance and scalable services. These examples demonstrate the industrial context in which software engineering and digital technologies are applied.

2.1 UCT Platforms and Technology Context

The supplied template lists several capabilities associated with UCT's platforms, including dashboard creation, analytics and reporting, alerts and notifications, third-party integration (for example Power BI, SAP and ERP systems) and a rule engine.

The template also refers to LoRaWAN-based solutions in areas such as Agritech, smart cities, industrial monitoring, smart street lighting and smart metering. It describes predictive maintenance using embedded systems, Industrial IoT and machine learning to estimate remaining useful life of machines.

For this internship project, the relevant learning connection is the use of Java to build a structured application. The Banking Information System does not attempt to reproduce UCT's industrial IoT platforms; instead, it is a Core Java project prepared within the internship framework.

2.1 Industrial Software Perspective

An industrial software solution must be evaluated beyond whether its basic functions execute. Important concerns include correctness, input validation, security, maintainability, performance, data persistence, recoverability and scalability.

The Banking Information System therefore separates account data, transaction behavior and menu operations into different methods and classes. This structure makes the prototype easier to test and extend.

Prototype limitation: The application stores data only in memory while the program is running. A production banking system would require a secure persistent database, strong authentication, authorization, encryption, audit controls, concurrency management and regulatory compliance.

2.2 About upskill Campus (USC)

The supplied internship template states that upskill Campus, along with The IoT Academy and in association with UniConverge Technologies, facilitated the internship process. The template describes USC as a career-development platform delivering personalized executive coaching in an affordable, scalable and measurable manner.

The template's workflow emphasizes exploring the problem statement, following project instructions and planning a solution, working on the project, checking improvements, validating implementation and performance, and finally submitting the project and report for certification.

The supplied sample also illustrates career-growth, professional-networking, collaboration and job/internship-oriented activities. These activities provide the broader context for the project work.

2.3 The IoT Academy

The supplied template describes The IoT Academy as the EdTech division of UCT and states that it runs long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship Program

- Gain practical experience of working in an industry-oriented environment.
- Solve a real-world-inspired software problem using programming concepts.
- Improve job prospects through practical project experience.
- Develop better understanding of the field and its applications.
- Improve communication, problem-solving and implementation skills.

2.5 Reference

1. Supplied Industrial Internship Report template provided for the internship submission format.
2. Java project source: BankingInformationSystem.java, prepared for this internship project.
3. GitHub repository links to be inserted after the repository is created and the files are uploaded.

2.6 Glossary

Term	Acronym / Meaning
Java	Object-oriented programming language used for the project
OOP	Object-Oriented Programming
JVM	Java Virtual Machine
PIN	Personal Identification Number
CRUD	Create, Read, Update and Delete operations
GitHub	Platform for hosting and versioning source code

3 Problem Statement

The project problem is to design and implement a simple Banking Information System using Core Java. The system should allow a user to create an account, authenticate using an account number and PIN, and perform basic banking operations.

Functional requirements

- Create a new bank account with name, phone number, PIN and opening balance.
- Authenticate an existing account before accessing account operations.
- Display account details and current balance.
- Deposit money with validation of positive amounts.
- Withdraw money only when sufficient balance is available.
- Transfer money between two existing accounts.
- Maintain an in-memory transaction history.
- Handle invalid numeric input and invalid menu selections without crashing.

Non-functional considerations

The program should be readable, modular, reasonably efficient for a small number of accounts and robust against common user-input errors. Security and persistent storage are identified as future requirements rather than claimed features of this educational prototype.

4 Existing and Proposed Solution

Existing approach

A very simple banking program can directly place all operations inside the main method. Such an approach may work for a demonstration, but it becomes difficult to maintain as features increase. It can also make validation, transaction handling and testing harder.

Proposed solution

The proposed solution uses separate Account and Transaction classes, a collection of accounts, dedicated methods for input validation and separate methods for account operations. A menu-driven interface provides a straightforward way to exercise the system.

Value addition

- Object-oriented organization improves readability and maintainability.
- Input validation reduces failures caused by invalid user input.
- Transaction history makes account activity visible during a session.
- Transfer operation demonstrates interaction between multiple account objects.
- The design leaves clear extension points for database storage, APIs and stronger security.

4.1 Code Submission (GitHub Link)

PLACEHOLDER — Replace YOUR_GITHUB_USERNAME with your GitHub username after creating the repository **upskillcampus**:

https://github.com/YOUR_GITHUB_USERNAME/upskillcampus/blob/main/BankingInformationSystem.java

4.2 Report Submission (GitHub Link)

PLACEHOLDER — After uploading this PDF to the same repository, update this report with the final working report URL and replace the old report file as instructed.

5 Proposed Design / Model

The application follows a simple layered flow: the user interacts with the console menu, the program validates input, the requested operation is performed on Account objects, and the resulting state is displayed back to the user.

5.1 High Level Diagram



Figure 1: High Level Diagram of the Banking Information System

5.2 Low Level Diagram

Account contains identity and balance data and maintains a list of Transaction objects. The main controller methods create accounts, authenticate users and call deposit, withdrawal, transfer and reporting operations.

5.3 Interfaces

The current interface is a text-based console. Inputs include account holder details, phone number, four-digit PIN, monetary amounts and menu selections. Outputs include account details, balances, success/failure messages and transaction history.

5.3 Detailed Data Flow

Step	Input	Processing	Output
1	Name, phone, PIN, opening balance	Validate input and create Account	New account number
2	Account number + PIN	Find account and authenticate	Login success/failure
3	Deposit amount	Validate amount and increase balance	Updated balance
4	Withdrawal amount	Check amount and balance	Updated balance / failure
5	Receiver + amount	Validate accounts and move amount	Transfer result
6	Transaction request	Read stored transaction objects	Transaction history

The implementation uses Java's LinkedHashMap to store accounts by account number and ArrayList to store transaction history. The choice keeps the code simple while supporting direct account lookup and ordered transaction display.

6 Performance Test

Performance testing for this prototype focuses on functional correctness, input robustness and the constraints of an in-memory console application. The supplied internship template specifically encourages consideration of memory, speed, accuracy, durability and power consumption where applicable.

6.1 Test Plan / Test Cases

ID	Test	Expected Result
TC01	Create account with valid data	Account is created and account number is displayed
TC02	Login with correct PIN	User enters account menu
TC03	Login with wrong PIN	Access is denied
TC04	Deposit positive amount	Balance increases
TC05	Withdraw within balance	Balance decreases
TC06	Withdraw above balance	Operation is rejected
TC07	Transfer to existing account	Sender decreases and receiver increases
TC08	Transfer to missing account	Operation is rejected
TC09	Enter invalid number	Program requests valid input
TC10	View transaction history	Stored transactions are displayed

6.2 Test Procedure

Compile the source using a Java Development Kit and run the class from a terminal. Start with account creation or use the included demonstration account (account number 1000, PIN 1234). Execute each menu operation using both valid and invalid values. Verify that the displayed balance changes match the expected operation and that invalid operations do not alter the balance.

6.3 Performance Outcome

For the intended educational workload, account lookup uses a hash-based map and is expected to be efficient for normal interactive use. Transaction history is stored in an ArrayList, so displaying all transactions for one account requires time proportional to the number of stored transactions.

The main constraint is persistence: all accounts and transactions are lost when the program exits. The prototype also does not measure production-grade throughput, latency, concurrent access, encryption or database durability. Those measurements would require a larger test environment and a persistent backend.

Important: The report does not claim laboratory performance numbers that were not actually measured. This avoids presenting fabricated benchmark results.

7 My Learnings

This project improved my understanding of how Java concepts can be combined into a complete application. I practiced classes and objects, encapsulation, collections, loops, conditional statements, methods, exception handling and input validation.

I also learned that software quality depends on more than successful execution. Invalid input, insufficient funds, missing accounts and authentication failures must be considered during design. The project improved my ability to break a larger problem into smaller modules and test each operation.

The internship experience also helped me understand the importance of documentation and version control. Maintaining the Java source and report in a GitHub repository creates a reproducible submission and makes the project easier to review.

8 Future Work Scope

- Connect the application to MySQL or another secure database for persistent storage.
- Replace plain PIN handling with secure password/PIN hashing and stronger authentication.
- Add role-based authorization for customers and administrators.
- Build a GUI or web interface instead of a console-only interface.
- Add account statements, beneficiary management and scheduled transactions.
- Implement transaction IDs, audit logs and better error reporting.
- Add automated unit and integration tests.
- Introduce concurrency controls and transactional database operations.
- Expose the banking functions through secure REST APIs.
- Add encryption, monitoring, backup and recovery mechanisms appropriate for a real financial system.

GitHub submission note: Create the repository with the exact name **upskillcampus**, upload **BankingInformationSystem.java** and this PDF, then replace the placeholder GitHub links in the report with the actual working links. The final report should be uploaded again after the links are updated so that the GitHub copy contains the current links.

GITHUB LINKS: MY NAME IS SAI YOGESH AND THIS IS MY PROJECT ENROLLED FOR CORE JAVA INTERSHIP AND PROJECT.

CODE LINK:

<https://github.com/expertX221/upskillcampus/blob/main/BankingInformationSystem.java>

REPORT LINK:

https://github.com/expertX221/upskillcampus/blob/main/BankingInformationSystem_Arjun_USC_UCT.pdf